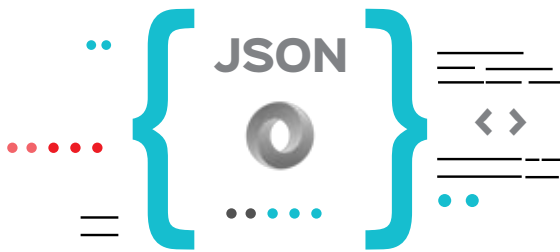


Carry Out JSONField Schema Validation Using JSON Models

This article shows how JSONField schema can be both manually written and automatically generated, and entails modifying the same in the latter case.



Among the numerous fields that exist for the models in the Django REST framework, JSONField is one where data is often required to abide by a strict format (or) schema, if included in the data used for testing generated APIs. Surely one can manually write the entire schema, but what if there was a way to skip writing it yourself and have it generated? This is where JSON models come to the rescue. In this article, we will be using example data, represented in snippets, on the software ‘Insomnia’, a popular platform that helps design and test APIs. We will cover two scenarios — writing the schema ourselves, followed by using JSON models to have the schema handed to us without much manual work.

Initial setup

Before getting familiar with the measures that can be taken for the validation of JSONField entry in Django REST framework (not bringing a defined schema into the equation just yet), it can be pointed out that in addition to other nonJSON fields, a POST operation can successfully be carried out if the following are included:

- An empty JSONField
 - A JSONField filled with various types of values
- The following examples will help understand this better.

models.py

```
from django.db import models
class Student(models.Model):
    student_name=models.CharField(max_length=100)
    roll_no=models.IntegerField()
    detailed_data=models.JSONField()
```

serializers.py

```
from rest_framework import serializers
from studentapp.models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model=Student

        fields='__all__'
```

views.py

```
from studentapp.serializers import StudentSerializer
from rest_framework.views import APIView
from rest_framework.response import Response
from django.http import JsonResponse,HttpResponse
from rest_framework import status

class StudentView(APIView):
    def post(self,request):
        serializer=StudentSerializer(data=request.
data)

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.
data,status=status.HTTP_201_CREATED)#inside if serializer.
is_valid()

            return Response(serializer.
errors,status=status.HTTP_400_BAD_REQUEST) #when serializer
is not valid
```

urls.py

```
from django.contrib import admin
from django.urls import path,include
from studentapp.views import StudentView
urlpatterns = [
    path('student/',StudentView.as_view()),
]
```